

# Assignment 12

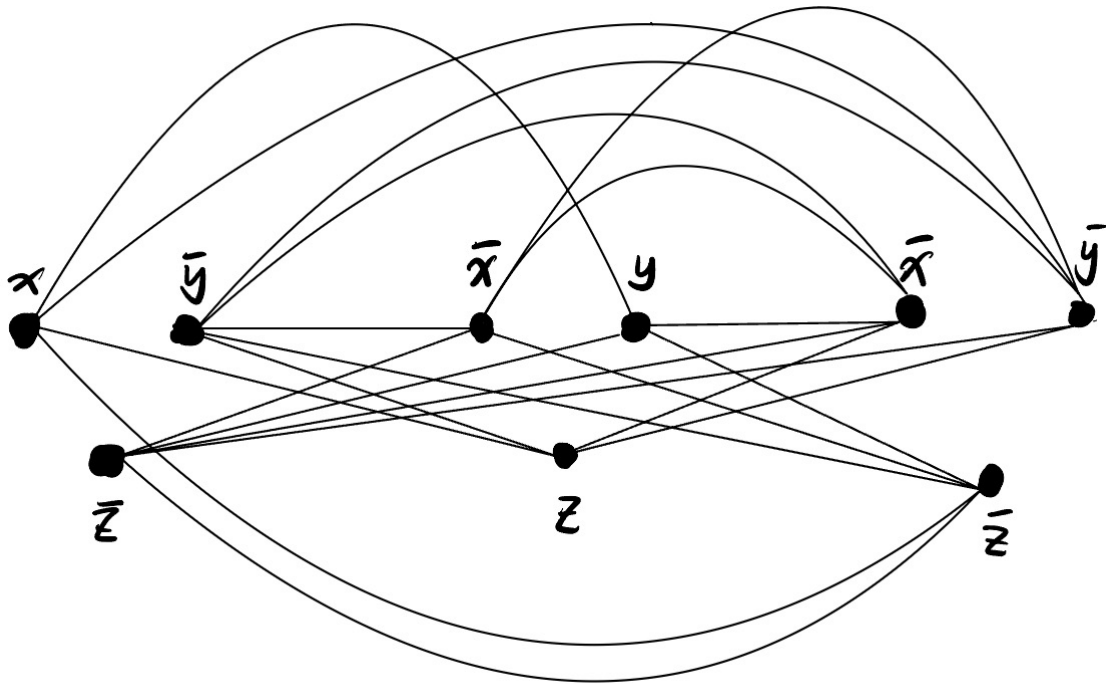
## Q1

Consider the following 3SAT formula:

$$\phi = (x \vee \bar{y} \vee \bar{z}) \wedge (\bar{x} \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$

Construct the reduction to a 3CLIQUE instance

## A1



The 3CLIQUE can be the clique that contains  $x$  in the first clause,  $y$  in the second clause and  $\bar{z}$  in the third clause.

## Q2

Show how to reduce  $2SAT \leq_p CLIQUE$ .

What can you conclude once you have a correct reduction?

## A2

Suppose  $\varphi$  is a 2SAT formula and

$$\varphi = (a_1 \vee b_1) \wedge (a_2 \vee b_2) \cdots \wedge (a_m \vee b_m)$$

where

$$a_i, b_i \in \{x_1, x_2, \dots, x_n, \bar{x}_1, \bar{x}_2, \dots, \bar{x}_n\}.$$

Then we will reduce 2SAT to CLIQUE by using the following rules:

1. Create a node for every literal in every clause of the Boolean formula.

2. Connect every two nodes if the two nodes are not in the same clause and the two nodes are not contradictory (i.e.  $\bar{a}$  and  $a$ ).

The claim is that  $\varphi$  is satisfiable iff  $G$  has a  $m$ -clique.

Now we prove its correctness.

First, suppose  $\phi$  is satisfiable. Then there is a truth assignment satisfying every clause. Since every clause is true, each clause  $C_i$  has at least one true literal. Choose one true literal from each clause. These chosen literals are pairwise non-contradictory, because they are all true under the same assignment. Therefore, the corresponding vertices are pairwise connected in  $G$ . Since we choose one vertex from each of the  $m$  clauses, these vertices form a clique of size  $m$ . Hence,

$$\langle G, m \rangle \in \text{CLIQUE}.$$

Conversely, suppose  $G$  has a clique of size  $m$ . Since edges are only added between vertices from different clauses, a clique can contain at most one vertex from each clause. Since there are  $m$  clauses and the clique has size  $m$ , the clique contains exactly one vertex from each clause.

Also, because every pair of vertices in the clique is connected by an edge, no two selected literals are contradictory. Therefore, we can assign all selected literals to be true consistently. Then every clause has at least one true literal, so  $\phi$  is satisfiable.

Therefore,

$$\phi \in \text{2SAT} \iff \langle G, m \rangle \in \text{CLIQUE}.$$

The construction creates  $2m$  vertices and at most  $O(m^2)$  edges, so it can be done in polynomial time. Hence,

$$\text{2SAT} \leq_p \text{CLIQUE}.$$

### Q3

Consider the following 2SAT formula:

$$\phi = (x \vee y) \wedge (\bar{x} \vee y) \wedge (\bar{x} \vee \bar{y}) \wedge (\bar{x} \vee \bar{y})$$

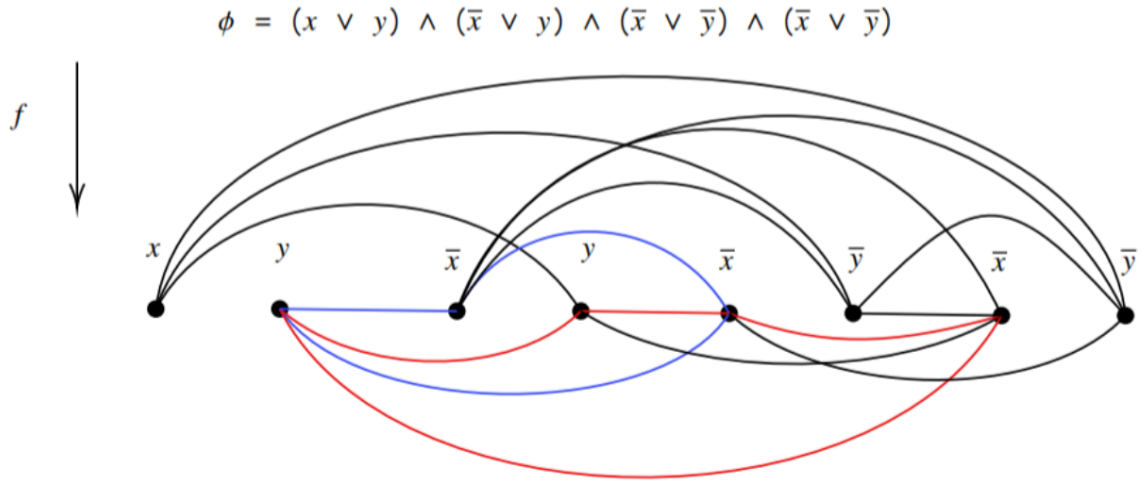
Construct the reduction to *CLIQUE* instance.

Does the obtained graph contain a 4CLIQUE?

Does it contain a 3CLIQUE?

Argue why.

**A3**



The figure above is the mapped graph, with blue-colored *3CLIQUE*. The *4CLIQUE* contains the red lines, as well as the two lines inside the red lines.

**Q4**

Let  $DOUBLE - SAT = \{ \langle \phi \rangle \mid \phi \text{ has at least two satisfying assignments} \}$ . Show that  $DOUBLE - SAT$  is NP-Complete.

**A4**

To demonstrate  $DOUBLE-SAT$  is NP-complete, we need to show that

1.  $DOUBLE-SAT$  is in  $NP$ .
2. Every  $A$  in  $NP$  is polynomial time reducible to  $DOUBLE-SAT$ .

First we prove  $DOUBLE-SAT$  is in  $NP$ . To illustrate  $DOUBLE-SAT$  is in  $NP$ , we need to show that there is a nondeterministic polynomial time Turing machine  $N$  that decides it.

$N =$  "On input  $\langle \varphi \rangle$ , where  $\varphi$  is a Boolean formula,

1. nondeterministic give two different boolean assignments.
2. If the result of both assignments are one, accept. Else reject."

Second, we prove that every  $A$  in  $NP$  is polynomial time reducible to  $DOUBLE-SAT$ .

According to Cook-Levin Theorem,  $SAT$  is NP-complete.

So we only need to show that

$$SAT \leq_p DOUBLE-SAT.$$

Next we will construct a polynomial time reduction  $f$  that maps an instance  $\varphi$  of  $SAT$  to an instance  $f(\varphi)$  of  $DOUBLE-SAT$ . Let

$$f(\varphi) = \varphi \wedge (x \vee \bar{x}),$$

where variable  $x$  doesn't appear in  $\varphi$ . If  $\varphi$  is satisfiable, then  $\varphi$  must have at least one satisfying assignment. So  $f(\varphi)$  must have at least two satisfying assignments since we can assign  $x$  to true or false. If  $\varphi$  isn't satisfiable,  $f(\varphi)$  isn't satisfiable as well. Thus,

$$SAT \leq_p DOUBLE-SAT$$

and *DOUBLE-SAT* is NP-complete.

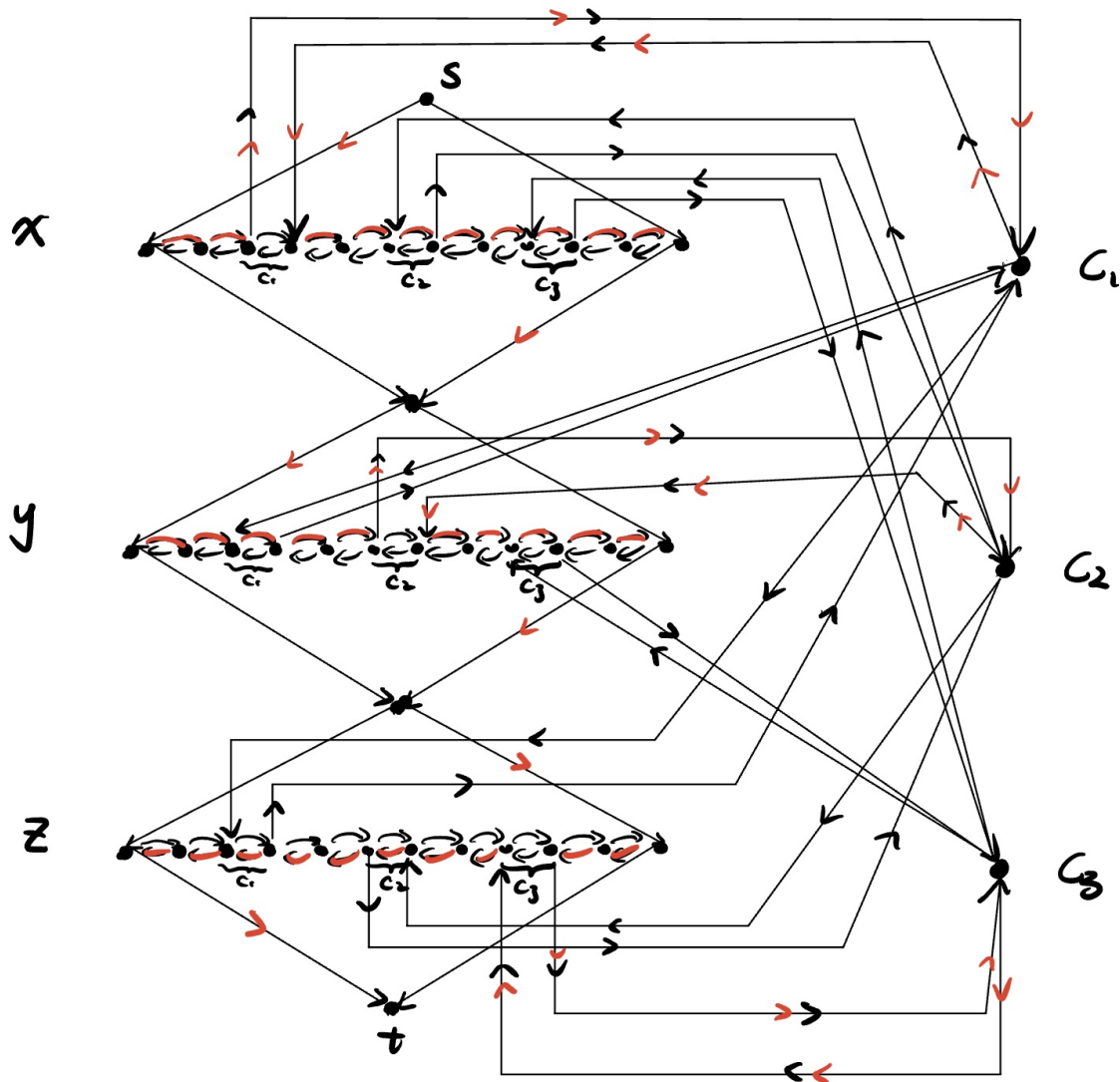
**Q5**

Consider the following 3SAT formula:

$$\phi = (x \vee \bar{y} \vee \bar{z}) \wedge (\bar{x} \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee z)$$

Construct the reduction to a *HAMPATH* instance and highlight the Hamiltonian path.

**A5**



**Q6**

A *colouring* of a graph is an assignment of colours to the nodes such that no node is adjacent to another node with the same colour.

$$3COLOR = \{ \langle G \rangle \mid G \text{ is colourable with 3 colors} \}.$$

Show that *3COLOR* is NP-Complete

## A6

To prove that *3COLOR* is NP-complete, we need to prove

1. *3COLOR* is in *NP*,
2. Every *A* in *NP* is polynomial time reducible to *3COLOR*.

First we prove *3COLOR* is in *NP*. To demonstrate *3COLOR* is in *NP*, we only need to show that there's an algorithm that can verify an instance of *3COLOR* in polynomial time. The algorithm

$M =$

"Given a graph  $G = (V, E)$ , non-deterministically guess a 3-coloring for each node.

Then for each edge  $e = \{v, w\}$ , if the color of  $w$  isn't different from the color of  $v$ , reject. Otherwise accept."

Obviously this can be done in polynomial time. So *3COLOR* is in *NP*.

To show that *3-COLOURING* is NP-hard, we give a polytime reduction from *3-SAT* to *3-COLOURING*.

That is, given an instance  $\phi$  of *3-SAT*, we will construct an instance of *3-COLOURING*, i.e. a graph  $G(V, E)$ , where  $G$  is 3-colourable iff  $\phi$  is satisfiable.

Let  $\phi$  be a *3-SAT* instance and  $C_1, C_2, \dots, C_m$  be the clauses of  $\phi$  defined over the variables

$$\{x_1, x_2, \dots, x_n\}.$$

The graph  $G(V, E)$  that we will construct needs to capture two things:

1. somehow establish the truth assignment for  $x_1, x_2, \dots, x_n$  via the colors of the vertices of  $G$  ;
2. somehow capture the satisfiability of every clause  $C_i$  in  $\phi$ .

To achieve these two goals, we will first create a triangle in  $G$  with three vertices

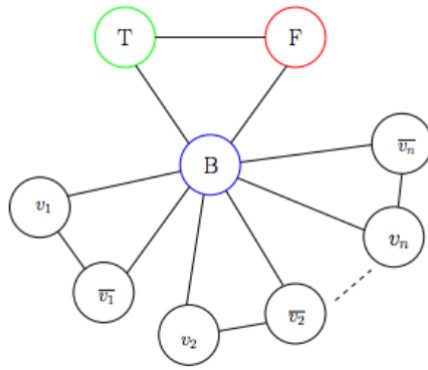
$$\{T, F, B\},$$

where  $T$  stands for True,  $F$  for False, and  $B$  for Base. Think of  $\{T, F, B\}$  as the set of colours we will use to colour, or label, the vertices of  $G$ .

Since this triangle is part of  $G$ , we already need 3 colours to colour  $G$ . We next add two vertices  $v_i, \overline{v_i}$  for every literal  $x_i$  and create a triangle

$$B, v_i, \overline{v_i}$$

for every  $(v_i, \overline{v_i})$  pair, as shown below.



Notice that so far, this construction captures the truth assignment of the literals. Since if  $G$  is 3-colourable, then either  $v_i$  or  $\bar{v}_i$  gets the colour  $T$ , and we just interpret this as the truth assignment to  $v_i$ .

Now we just need to add constraints, such as edges or extra vertices, to  $G$  to capture the satisfiability of the clauses of  $\phi$ . To do so, we introduce the **OR-gadget**.

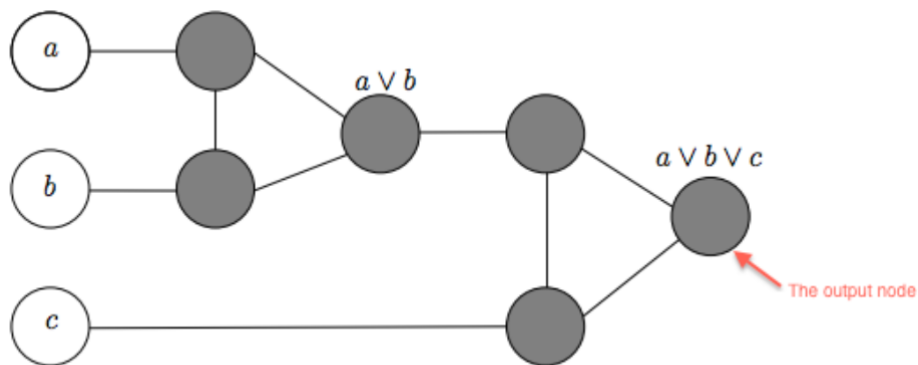
For a clause

$$C_i = (a \vee b \vee c),$$

we need to express the OR of its literals using our colours

$$\{T, F, B\}.$$

We achieve this by creating a small gadget graph that we connect to the literals of the clause. The OR-gadget is constructed as follows:



But what is it actually doing? You can think of this gadget graph as a circuit whose output is the node labeled  $a \vee b \vee c$ . We basically want this node to be coloured  $T$  if  $C_i$  is satisfied and  $F$  otherwise. Notice that this is a two step construction: The node labeled  $a \vee b$  captures the output of  $(a \vee b)$  and we repeat the same operation for  $(a \vee b) \vee c$ .

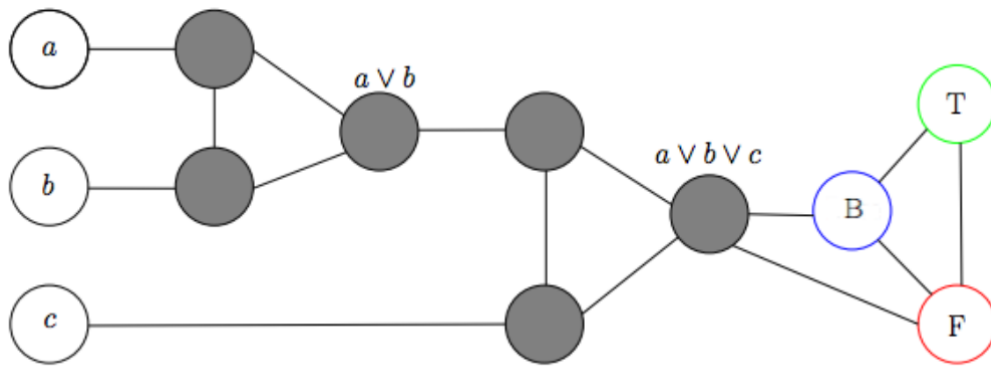
If you play around with some assignments to  $a, b, c$ , you will notice that the gadget satisfies the following properties:

1. If  $a, b, c$  are all coloured  $F$  in a 3-colouring, then the output node of the OR-gadget has to be coloured  $F$ . Thus capturing the unsatisfiability of the clause

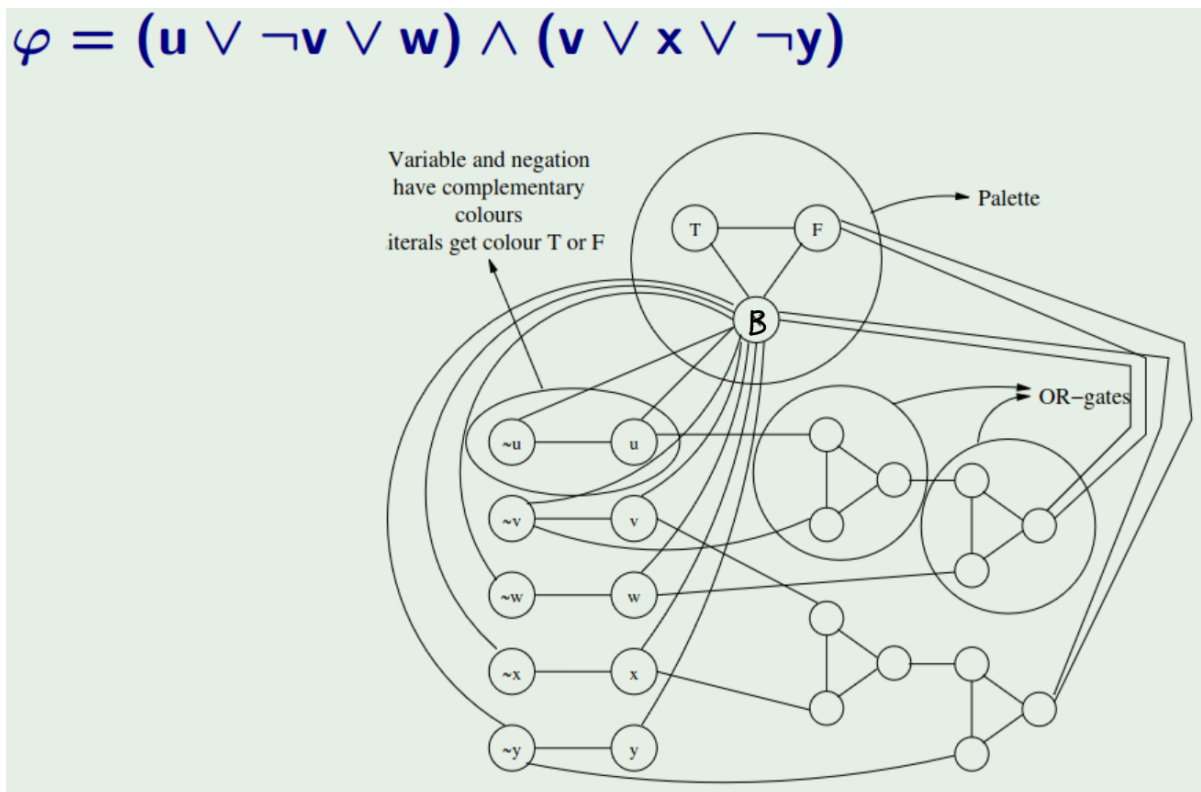
$$C_i = (a \vee b \vee c).$$

2. If one of  $a, b, c$  is coloured  $T$ , then there exists a valid 3-colouring of the OR-gadget where the output node is coloured  $T$ . Thus again capturing the satisfiability of the clause.

We're almost done our construction. Once we add the OR-gadget of every  $C_i$  in  $\phi$ , we connect the output node of every gadget to the Base vertex and to the False vertex of the initial triangle, as follows:



Here is a complete example:



Now we prove the correctness,

1.  $\varphi$  is satisfiable implies  $G$  is 3-colorable:

- If variable  $x_i$  is assigned True, color the node  $x_i$  True and  $\overline{x_i}$  False.
- For each clause  $C_j = (a \vee b \vee c)$ , at least one of  $a, b, c$  is colored True. OR-gadget for  $C_j$  can be 3-colored such that output is True.

2.  $G$  is 3-colorable implies  $\varphi$  is satisfiable:

- If the node  $x_i$  is colored True, then set variable  $x_i$  to be True. This is a legal truth assignment.
  - Consider any clause  $C_j = (a \vee b \vee c)$ . It cannot be that all  $a, b, c$  are False. If so, output of OR-gadget for  $C_j$  has to be colored False, but output is connected to 'Base' and 'False' in the graph.